Chapter 9: Building Internet Applications

In this chapter, you will learn to build applications that access the Internet.

The chapter begins with an overview of Internet concepts and terminology. It then describes how to use the Internet Explorer control and the ActiveX Web Browser control in your applications. The later sections discuss the different categories of classes provided by Microsoft Foundation Class (MFC) and explain how to build custom applications with these classes.

Objectives

After completing this chapter, you will be able to:

- Describe the Internet framework.
- Describe the different types of Internet applications.
- Create MFC-based applications that invoke Internet Explorer.
- Use the Web Browser control in MFC-based applications.
- Create MFC-based applications that use the WinInet classes to communicate across the Internet.
- Create MFC-based applications that use the synchronous and asynchronous WinSock classes.

273 Basic Internet Concepts

This section introduces the basic Internet concepts and terminology that you will need to successfully complete this chapter. It also describes the range of Internet capabilities an application can offer, and the MFC support available for implementing Internet applications.

This section includes the following topics:

- The Internet Framework
- Types of Internet Applications
- MFC Support for Internet Applications

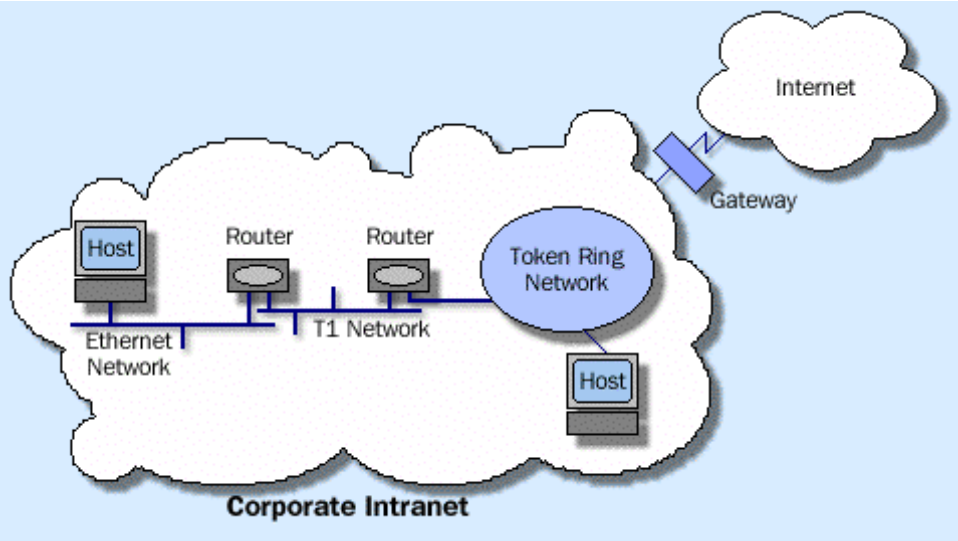
Notes

274 The Internet Framework

A network is a collection of computers that are connected by a data-link medium, such as Ethernet or Token Ring. The Internet is a global network of such computer networks. An intranet is an organization-level internet.

While each network has its own method of communicating among its host computers, there is a need to standardize communication among all networks on the Internet. The TCP/IP protocol suite is the standard protocol that enables communication on the Internet.

The following illustration shows how different networks communicate with each other. To display the definition for a term, click the term's label in the illustration. (CD-ROM displays context-sensitive text.)



The purpose of any network is to transfer data between host computers. The data is meaningful only to the applications residing on the hosts; the network itself sees it only as data. The TCP/IP protocols package the host computer's data and add header information to enable routing of the data.

You can think of a program's interaction with a network as the activation of a series of layers, starting at the application layer and ending at the network's physical layer, where the data is sent as a stream of bits over the network. To see an animation that describes the communication process, click this icon.

(CD-ROM plays the animation, "Internet Communication.")

Notes

275 Types of Internet Applications

The Internet and intranets are becoming more and more important to application developers. Not only are developers using the Internet to retrieve resources and to communicate with peers, but they are also finding it necessary to connect their applications to the Internet.

An application can connect to or communicate with the Internet in various ways. In the simplest case, an "Internet-aware" application recognizes a reference to an Internet resource and calls another application, such as Internet Explorer, to handle all Internet access. "Internet-enabled" applications interact directly with the Internet using low-level techniques.

Internet-Aware Applications

An Internet-aware application accesses the Internet indirectly, through a software proxy. The application invokes this proxy, often a browser such as Internet Explorer, or a utility such as Telnet, as a separate process. After completing the task associated with the Internet resource, the user closes the proxy and continues with the main application. From a developer's perspective, this solution provides a great deal of functionality and is easy to implement. On the other hand, the application has very little control over the proxy utility; it starts the utility, and has no more association with it.

Internet-Enabled Applications

An Internet-enabled application interacts directly with the Internet. The application can interact with the network software at a high level, through an embedded control such as the Web Browser control, or through application-level services such as File Transfer Protocol (FTP) and Hypertext Transfer Protocol (HTTP). If the application's requirements call for a closer interaction with the TCP/IP protocol suite, you can make calls to the WinSock API functions.

Notes

276 MFC Support for Internet Applications

In the Visual C++ and MFC environment, you can use the Web Browser control and the MFC Internet classes to build Internet applications.

The browser object is available as a control that you can embed within the application. You also can control Internet Explorer through Automation. Both these methods use the HTTP protocol for viewing HTML pages, which provide the basis for the World Wide Web.

Alternatively, you can use the **CHTMLView** class to implement the functionality of the browser control in your applications. The **CHTMLView** class is derived from **CView** and is a wrapper class for the **Web Browser** control.

MFC also provides some control classes, such as **CIPAddressControl** and **CProgressCtrl**, that you can use to build Internet-enabled applications. For more information about these control classes, see Internet Explorer 4.0 Common Controls in Chapter 5: Using Controls.

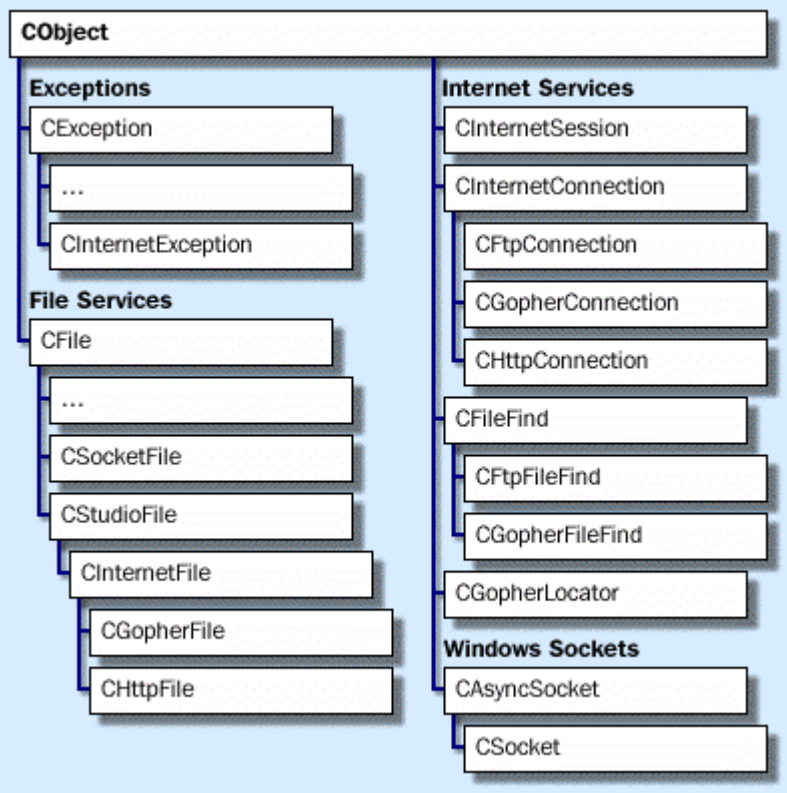
At a lower level, MFC includes Win32 Internet (WinInet) class support for the HTTP protocol through the **CInternetSession** and **CHttpConnection** classes.

At the same level as HTTP support, classes are also available to implement custom FTP and Gopher services.

Below the level of HTTP support, several classes are available to interact with the WinSock API. These classes can be used to create custom service applications, such as an application that offers features that the FTP, HTTP, or Gopher classes do not have.

Notes

The following illustration shows the Internet classes provided by MFC.



Notes

277 Using the Internet Explorer Object

The Internet Explorer object is an Automation object that you can use to control Internet Explorer from within an application. This section introduces the object model and describes how to use it in your application.

This section includes the following topics:

- Properties and Methods
- Controlling Internet Explorer
- Using Automation

Notes

278 Properties and Methods

The Internet Explorer object has a predefined set of properties and methods. It does not support events.

Properties

Properties are settings or values that are maintained by an object. An object exposes its **Get** member functions to enable your application to retrieve the value of properties. For writable properties, the object exposes the **Set** member functions.

The following table lists some of the properties exposed by the Internet Explorer object.

Function	Description
BOOL GetBusy()	Returns **TRUE** if the object is engaged in a navigation or downloading operation.
void SetVisible(BOOL val)	Accepts a BOOL value and controls whether the object is visible.
void SetFullScreen(BOOL val)	If the value is **TRUE**, Internet Explorer will be placed in full-screen mode. In full-screen mode, the Internet Explorer main window is maximized, and the status bar, menu bar, and title bar are hidden.

Methods

A method enables your application to request an action of the object. The following table lists some of the methods available for the Internet Explorer object.

Method	Description
void GoBack()	Tells the object to navigate backward one item in the history list.
void GoForward()	Tells the object to navigate forward one item in the history list.
void GoHome()	Tells the object to navigate to the home page specified in the **Internet Explorer Options** dialog box.
void GoSearch()	Tells the object to navigate to the search page as specified in the **Internet Explorer Options** dialog box.

For a complete listing of properties and methods, search for "CHTMLView class members" in MSDN Library Visual Studio 6.0.

For more information about the Internet Explorer object model specification, go to the Win32 Internet Programmer's Reference page by clicking on this icon.

 (CD-ROM links to the Web site: www.microsoft.com/msdn/sdk/mactivex/docs/wininet/inetref.htm.)

Notes

279 Controlling Internet Explorer

There are two ways you can invoke the Internet Explorer: use the **ShellExecute** function or use Automation.

Using the ShellExecute Function

If your application needs only to start Internet Explorer (or your default browser) and has no need to interact with it, you can invoke the browser directly by calling the **ShellExecute** function. To invoke **ShellExecute**, implement a simple command handler like the one shown in the following example:

```
void CMyView::OnUtilityLaunchBrowser()
{
    ShellExecute(MyWin,"open","c:\\ie\\iexplore.exe", NULL, NULL, SW_SHOW);
}
```

Using Automation

If your application needs to control Internet Explorer after starting it, you can use Automation to set properties and invoke methods of the **Internet Explorer** object. Using Automation gives your application a much greater degree of control over Internet Explorer than does invoking it as an independent process through the **ShellExecute** function.

▶ **To use Automation with the Internet Explorer object**

1. Add Automation support for clients to your application.
2. Use ClassWizard to define a new class based on the Internet Explorer object model.
3. Create an instance of the class within your code.
4. Attach a dispatch object to the class.

   A dispatch object exposes the IDispatch interface. Through the IDispatch interface, a client application sets and gets properties of an object and calls methods of an object.
5. Manipulate the object through its properties and methods, using member functions of the created class.

Notes

280

Using Automation

The steps in implementing Automation are described below.

Adding Automation Support for Clients

You can add Automation support to clients by using either of the following two methods:

- Select the **Automation** check box in Step 3 of MFC AppWizard.

  This will automatically include the code required for automation support in a new application.
- Manually add code to provide automation support for clients.

  Use this method to upgrade an existing application.

▶ **To manually add Automation support for clients to an existing application**

1. Add the include directive for the file afxdisp.h to Stdafx.h as follows:

   ```
   #include <afxdisp.h>
   ```

2. Add the following line of code to the **InitInstance** function of your **CWinApp**-derived class. This function initializes the OLE dynamic libraries.

   ```
   AfxOleInit();
   ```

Using ClassWizard to Create the Internet Explorer Object Class

Using the information contained in type library files, ClassWizard generates a wrapper class containing member functions for each method and property.

▶ **To create a wrapper class for the Internet Explorer object**

1. Start ClassWizard.
2. In the **ClassWizard** dialog box, click **Add Class**, and then click **From a type library**.
3. In the **Import from Type Library** dialog box, navigate to the \Windows\System (or System32) folder and select **Shdocvw.dll**.
4. From the list that appears, select **IWebBrowserApp**, and then click **OK**.

   ClassWizard will create a new class for your project called **IWebBrowserApp**.

Creating an Instance of Internet Explorer

After adding the class definition for the **Internet Explorer** object, create an instance of the **Internet Explorer** object. First, make sure that you have included the header file Shdocvw.h in all the implementation files that access the object's properties and methods. Then add code as follows to create an instance of the object:

```
IWebBrowserApp* m_pIE = new IWebBrowserApp;
```

Notes

Attaching a Dispatch Interface

When ClassWizard creates a new class by importing a type library, it derives the new class from **COleDispatchDriver**. The **COleDispatchDriver** class implements the client-side support for applications that support Automation. Member functions of this class provide access to attach, detach, create, and release a dispatch connection of type **IDispatch**.

To attach an **IDispatch** connection to your class, call the **CreateDispatch** member function. There are two variations of this function: One requires a class identifier (CLSID) of the Automation object; the other requires the programmatic identifier (ProgID), an easy-to-remember string used to identify the object. The following example uses the ProgID to attach a connection to your class:

```
m_pIE->CreateDispatch("InternetExplorer.Application");
```

Using the Object

Once the object is created and has an **IDispatch** connection to your code, your application can set properties and invoke methods of that object. The following example code causes Internet Explorer to navigate to a specific URL site, and makes the browser visible:

```
COleVariant noArg;  //  declare a VT_EMPTY variant.
m_pIE->Navigate("www.microsoft.com", &noArg, &noArg, &noArg, &noArg);
m_pIE->SetVisible(TRUE);
```

The empty **COleVariant** arguments can be set to further customize the **Navigate** method.

For more information about **COleVariant** arguments, search for "COleVariant::COleVariant" in MSDN Library Visual Studio 6.0.

Notes

281 Using the Web Browser Control

The **Web Browser** control is an ActiveX component available in Visual Studio. This control encapsulates the browsing capabilities of Internet Explorer, enabling you to create sophisticated HTTP browsing applications with little effort.

The **Web Browser** control supports Web browsing through both point-and-click hyperlinking and Uniform Resource Locator (URL) navigation. The control maintains a history list that enables the user to browse forward and backward through previously browsed sites, folders, and documents. The **Web Browser** control directly handles the navigation, hyperlinking, history lists, favorites, and security.

In this section, you will learn how to use the **Web Browser** control in MFC-based applications.

This section includes the following topics:

- Properties, Methods, and Events
- Adding the Web Browser Control
- Creating the Web Browser Control
- Implementing the Web Browser Control
- Implementing HTML Views

Notes

282 Properties, Methods, and Events

The **Internet Explorer** object and the **Web Browser** control share a common object model. Most of the properties and methods are common to both the objects.

For more information about the properties and methods of the **Web Browser** control, see Properties and Methods in this chapter.

The main difference between the **Internet Explorer** object and the **Web Browser** control is that the browser control also supports events. You can override these events to implement the browsing requirements of your applications.

Events

Events are notifications that a control can send to a control container in response to user interaction.

The following table lists some **Web Browser** functions that you can override to take appropriate action for your application.

Function	Description
void ProgressChange(long Progress, long ProgressMax)	Occurs when the progress of a download operation is updated. The first parameter specifies the amount of total progress or –1 when the download is complete. The second parameter specifies the maximum progress value.
void StatusTextChange(LPCTSTR text)	Occurs when the status bar text has changed.
void TitleChange(LPCTSTR text)	Occurs when the title of the current document becomes available or changes.

For a complete list of properties, methods, and events, search for "CHTMLView class members" in MSDN Library Visual Studio 6.0.

Notes

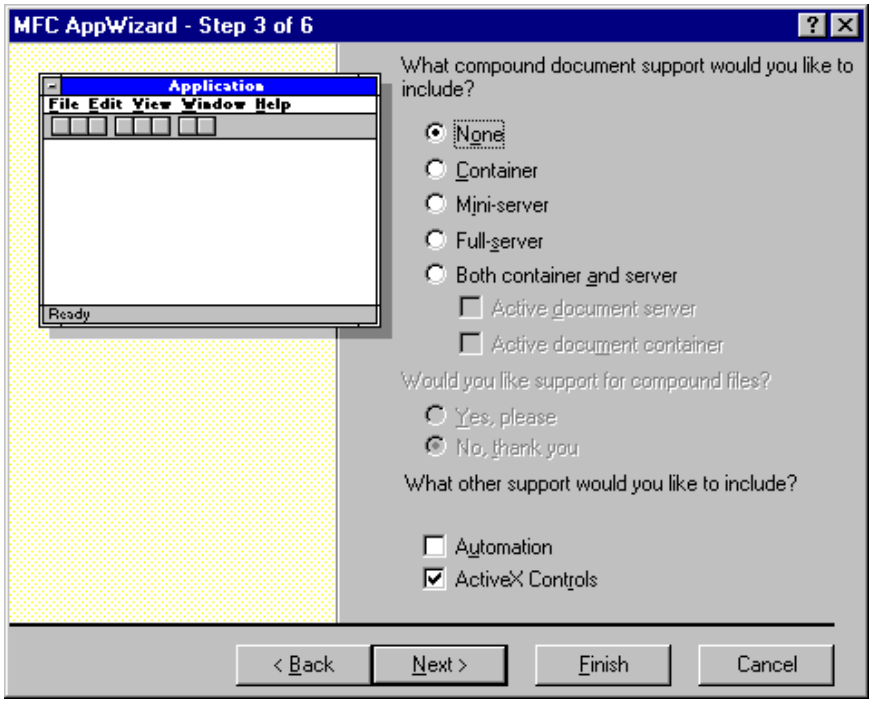
283 Adding the Web Browser Control

The first step in creating an application with a Web Browser control is to create an application that can act as an ActiveX control container. You can do this using one of the following methods:

- In Step 3 of AppWizard, select the **ActiveX Controls** check box.

  To see an illustration of the **MFC AppWizard - Step 3** dialog box with the check box selected, click this icon.

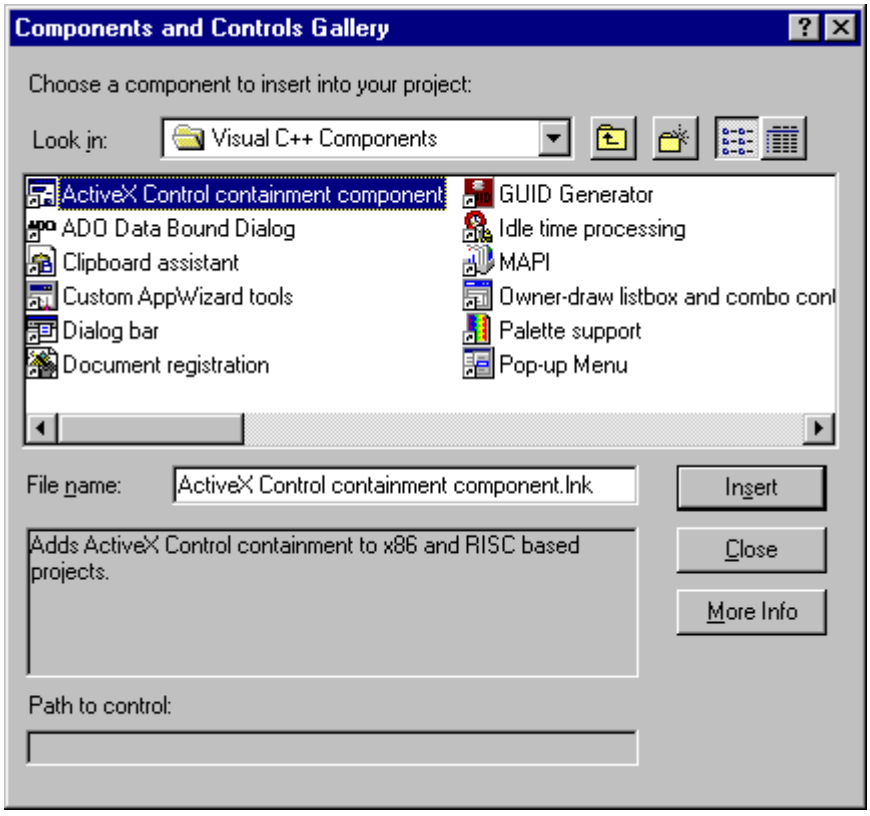
  (CD-ROM displays the following illustration.)



Notes

- Use the Components and Controls Gallery to insert the ActiveX Control containment component into your project.

  To see an illustration of the **Components and Controls Gallery** dialog box, click this icon. (CD-ROM displays the following illustration.)



- Manually add the following lines of code to your application.

  1. Add this line to the Stdafx.h header file.

     ```
     #include <afxdisp.h>
     ```

  2. Add the following line to the **InitInstance** member function of the **CWinApp**-derived class.

     ```
     AfxEnableControlContainer();
     ```

     The **AfxEnableControlContainer** function enables support for OLE controls.

Once you have added control container support to your application, you can add the **Web Browser** control to your project.

Notes

▶ **To add the Web Browser control to your project**

1. On the Visual Studio **Project** menu, click **Add To Project**, and then click **Components and Controls**.
2. Double-click the **Registered ActiveX Controls** folder.
3. Select **Microsoft Web Browser** and click **Insert**, and then click **OK** to confirm the selection.

In the ClassView pane, you see a class named **CWebBrowser2**. Expand this class to examine the properties and methods exposed by this control.

To view a demonstration of adding the **Web Browser** control to an application, click this icon.
 (CD-ROM plays the demonstration, "Adding the Web Browser Control to an Application.")

If you want your application to respond to events, you must manually add event handlers. For more information about how to add event handlers, see Implementing the Web Browser Control in this chapter.

Notes

284 Creating the Web Browser Control

After installing the **Web Browser** control into your project as a wrapper class, you need to construct an instance of the class and then create the control. To do this, you complete the following three steps:

1. Add include directives for the **Web Browser** control's header file to the appropriate source files.
2. Create a member variable that will point to the **Web Browser** control object in the view class.
3. Add the code to create the control.

These steps are described in more detail below.

▶ **To create an instance of the Web Browser control**

1. In the header file of the view class, add the following line after the include directive for Stdafx.h:

   ```
   #include "webbrowser2.h"
   ```

2. In the Visual Studio ClassView pane, right-click the view class and click **Add Member Variable**.

   The **Add Member Variable** dialog box appears.

3. In the **Variable Type** field, type **CWebBrowser2***.
4. In the **Variable Name** field, type **m_pBrowse**. Accept the default setting of public access and click **OK**.
5. It is good practice to initialize the member variable to NULL. Add the following line to the view constructor:

   ```
   m_pBrowse = NULL;
   ```

6. Create a resource symbol named IDC_WBC. You need this resource ID to create the Browser control.
7. Use ClassWizard to add a message handler for WM_CREATE to the view class.

Notes

8. In the **OnCreate** member function of the view class, after the call to the base class's **OnCreate** function, add code to create the control. To see sample code that shows the complete **OnCreate** function, click this icon.


(CD-ROM displays the following sample code.)

```
// Creating the Web Browser Control using a dummy menu ID
// Samples\CH09\Browse
int CBrowseView::OnCreate(LPCREATESTRUCT lpCreateStruct)
{
    if (CView::OnCreate(lpCreateStruct) == -1)
        return -1;

    // Define the area where the control will reside.
    CRect rect;
    GetClientRect(&rect);
    //
    // Create the control.
    // IDC_WBC is a unique identifier for the control and was defined
    // using a dummy menu item.
    m_pBrowse= new CWebBrowser2;
    ASSERT(m_pBrowse);
    if (!m_pBrowse->Create(NULL,NULL, WS_VISIBLE,rect,this, IDC_WBC)) {
        TRACE("failed to create browser\n");
        return 0;
    }
    // Initialize the first URL.
    COleVariant noArg;
    m_pBrowse->Navigate("www.microsoft.com",&noArg,&noArg,&noArg,&noArg);

    return 0;
}
```

Note Although you can successfully build the project with this code in it, you will not see the **Web Browser** control until you add code to implement it.

Notes

285 Implementing the Web Browser Control

Implementing a Web Browser control involves three main steps:

- Set properties
- Call methods
- Handle events

To see a demonstration of how to implement a Web Browser control in the view class of an application, click this icon.

 (CD-ROM plays the demonstration, "Using the Web Browser Control in an Application.")

Setting Properties

The following example sets the properties of the **Web Browser** control to adjust the control's size when handling a WM_SIZE message from Windows:

```
void CBrowseView::OnSize(UINT nType, int cx, int cy)
{
    // the base class handler is called.
    CView::OnSize(nType, cx, cy);

    // m_pBrowse is a member variable of the view class
    if (m_pBrowse)
    {
        m_pBrowse->SetWidth(cx);
        m_pBrowse->SetHeight(cy);
    }
}
```

Invoking Methods

Invoking a method of the **Web Browser** control is similar to invoking an ActiveX control method. You begin by identifying the methods that you need to implement based on the requirements of your application.

The following example shows how to use the **Navigate** method of the **CWebBrowser** class.

```
COleVariant noArg;
// navigate to a specific home page.
m_pBrowse->Navigate("www.microsoft.com", noArg, noArg, noArg, noArg);
```

You can then build and run your application. You will see that the **Web Browser** control displays the Web site specified in the **Navigate** method.

Notes

Handling Events

Events are not automatically included in the **Wrapper** class when you add the class by using the Visual Studio Components and Controls Gallery. To support events, you need to add code manually.

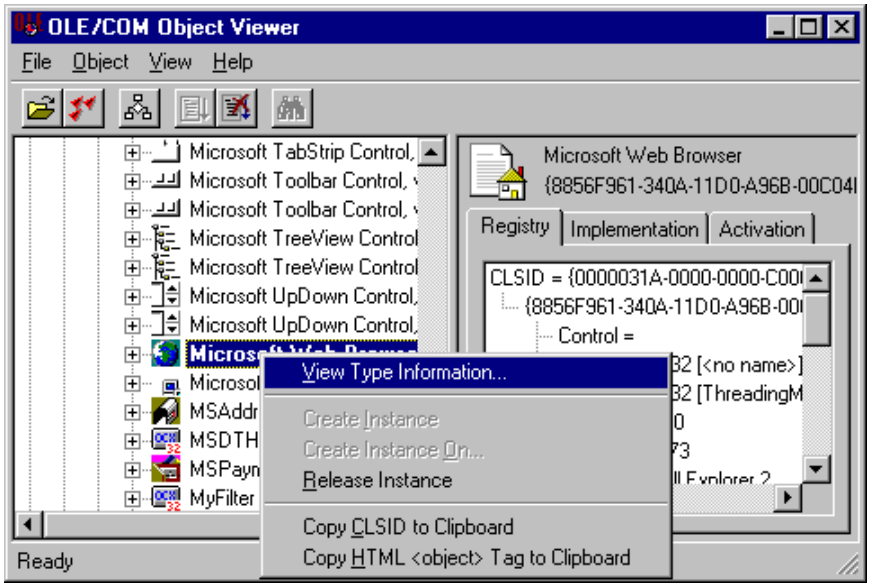
▶ **To handle an event**

1. Determine the function prototype and ID of the event.
2. Add the event function handler to the **Wrapper** class.
3. Link the event to the handler.

The following procedures describe the steps for adding the **TitleChange** event to the **Wrapper** class in the sample application Browse, which is located in the folder \Samples\CH09.

▶ **To determine the prototype and id for the TitleChange event handler**

1. On the Visual Studio **Tools** menu, click **OLE/COM Object Viewer**.
2. In the left pane of the **OLE/COM Object Viewer** dialog box, double-click the **Controls** folder and right-click **Microsoft Web Browser**.
3. On the shortcut menu, click **View Type Information**. To see an illustration of this step, click this icon. (CD-ROM displays the following illustration.)



The **ITypelib Viewer** dialog box appears.

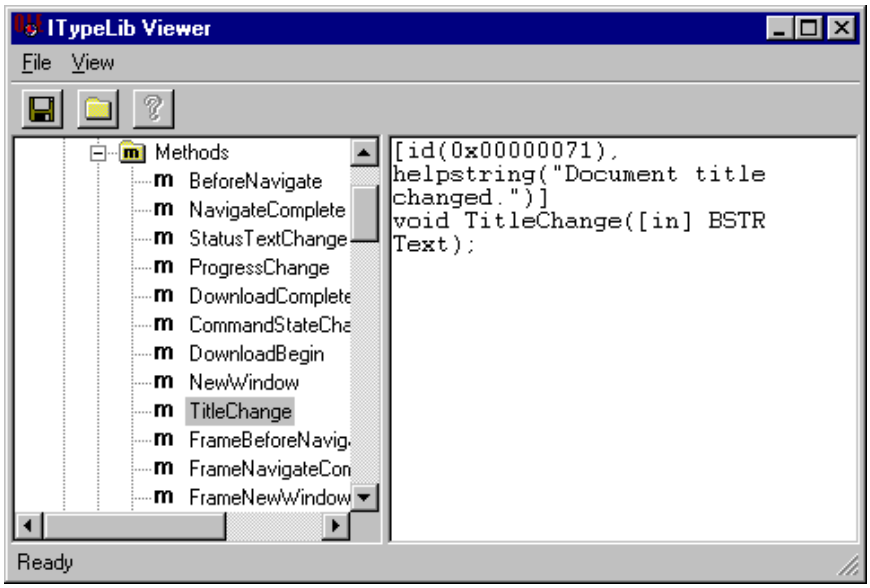
4. In the **ITypeLib Viewer** dialog box, double-click **dispinterface DWebBrowserEvents**. Double-click the **Methods** folder to display the events for the **Web Browser** control.

Notes

5. In the **Methods** folder, click **TitleChange**.

   The information you require about the prototype and ID for the event handler is displayed in the right of the **ITypeLib Viewer** dialog box. To see an illustration of the **ITypeLib Viewer** dialog box, click this icon.

   (CD-ROM displays the following illustration.)



Adding the Event Handler

The prototype of the **TitleChange** event, which is written in Microsoft Interface Definition Language (MIDL), is as follows:

```
void TitleChange([in] BSTR Text)
```

The following code shows the MIDL syntax translated to C++:

```
void ClassName::OnTitleChange(LPCTSTR Text)
```

Once you know the prototype, you add the event handler as you would add any other function. In the Visual Studio ClassView pane, right-click the class name that you want to handle the event. Then click **Add Member Function** on the shortcut menu and provide the prototype information.

Here is an example of the code for the event **OnTitleChange**:

```
void CBrowseView::OnTitleChange(LPCTSTR Text)
{
// Format a string and set the mainframe window's title
CString strTitle;
strTitle.Format("MyBrowser - %s", Text);
AfxGetMainWnd()->SetWindowText(strTitle);
}
```

Notes

Linking the Event to the Handler

Finally, you must link the event to the event handler by adding an event map entry.

▶ **To make an event map entry**

1. Add a DECLARE_EVENTSINK_MAP() macro call to the header file of the class that handles the event.
2. Add an include directive for Afxdisp.h in the implementation file of the class that handles the event.
3. Add an event sink map in the implementation file of the class that handles the event.

The following procedure describes these steps for the sample application Browse.

▶ **To add a DECLARE_EVENTSINK_MAP() macro call in BrowseView.h**

1. In the file BrowseView.h, add the following line after the call to `DECLARE_MESSAGE_MAP():`

   ```
   DECLARE_EVENTSINK_MAP()
   ```

2. In the implementation file of the class that handles the event, add the following line after the directive #include "Stdafx.h":

   ```
   #include "afxdisp.h"
   ```

3. Add an event sink map after the line END_MESSAGE_MAP as follows:

   ```
   BEGIN_EVENTSINK_MAP(CBrowseView, CView)
           ON_EVENT(CBrowseView, IDC_WBC, 0x71, OnTitleChange, VTS_BSTR)
   END_EVENTSINK_MAP()
   ```

   The value 0x71 is the ID value you retrieved when you determined the prototype and ID for the **TitleChange** event handler.

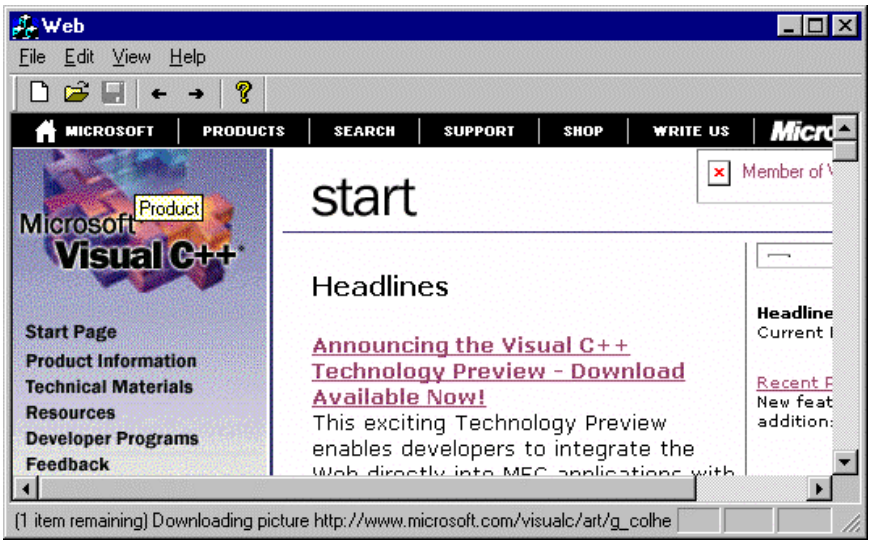
Compile and run the application to see the effect of implementing the **TitleChange** event.

Notes

287 Implementing HTML Views

The **CHTMLView** class is derived from **CView** and provides specialized functionality for designing applications that are dependent on the Web. **CHTMLView** provides the functionality of the **Web Browser** control within the context of the MFC document/view architecture.

To see an illustration that shows an application with an HTML view, click this icon.
 (CD-ROM displays the following illustration.)



The HTML view is available only to applications running under Windows 95 and Windows NT versions 3.51 and later in which Internet Explorer 4.0 (or later) has been installed.

▶ **To create an application (either SDI- or MDI-based) with an HTML view**

- Invoke MFC AppWizard. In Step 6 of 6, set the application's base view class to **CHTMLView**.

▶ **To convert an existing application to support an HTML view**

- Open the implementation file of the view class, and replace all instances of your current view class (usually **CView**) with **CHTMLView**.
- Repeat the same substitution in your view's header file, replacing your current view class with **CHTMLView**.
- In your application's StdAfx.h file, append the following line of code:

```
#include <afxhtml.h>
```

Notes

▶ To implement an application with an HTML view

When you create an HTML view application using MFC AppWizard, AppWizard automatically adds the code to implement the view.

To implement HTML view manually, you need to override the **OnInitialUpdate** member function of your application's view class. In the function body of **OnInitialUpdate**, add a call to the **CHTMLView::Navigate** function to cause the contained **Web Browser** control to load the specified HTML page.

For example, the following line of code causes your application to start with Microsoft's home page displayed in its view (assuming you are connected to the Internet when you start the application).

```
Navigate( _T("http://www.microsoft.com"),0,0,0);
```

For more information about HTML view, search for "CHTMLView" in MSDN Library Visual Studio 6.0.

Notes

288 Using the WinInet Classes

This section provides an overview of the three important Internet protocols: HTTP, FTP, and Gopher. It also describes the MFC classes that support these protocols.

This section includes the following topics:

- Connecting to the Internet
- Writing HTTP Applications
- Writing FTP Applications
- Writing Gopher Applications

Notes

289 Connecting to the Internet

An Internet session is your link with a service, such as HTTP, FTP, or Gopher. You can have multiple connections with any of the services. The MFC classes **CInternetSession**, **CInternetConnection**, and **CInternetFile** provide support for HTTP, FTP, and Gopher connections.

CInternetSession

Use the **CInternetSession** class to create and initialize one or more simultaneous Internet sessions. The first step in creating an Internet link is to construct a **CInternetSession** object as follows:

```
CInternetSession session;
```

Then, call **CInternetSession::OpenUrl** to retrieve a file handle to an FTP, HTTP, Gopher, or FILE resource. The type of handle retrieved depends on the URL type. The following table lists the valid types.

URL type	File handle type
file://	**CStdioFile***
http://	**CHttpFile***
ftp://	**CInternetFile***
gopher://	**CGopherFile***

If you require a specific service, you can retrieve the handle using one of the following functions:

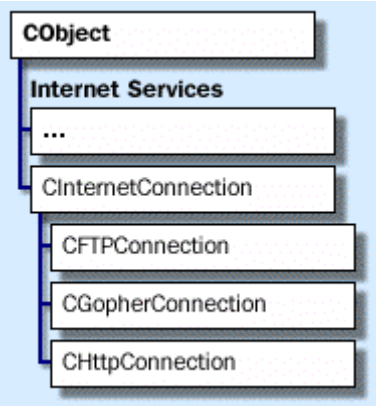
- **CInternetSession::GetHttpConnection**
- **CInternetSession::GetFtpConnection**
- **CInternetSession::GetGopherConnection**

CInternetConnection

The **CInternetConnection** class manages your connection to an Internet server. **CInternetConnection** is the base class of the classes that provide functionality for communicating with an FTP, HTTP, or Gopher server.

Notes

The following illustration shows the classes derived from **CInternetConnection**.



If the client computer is directly connected to the Internet, call ***GetXxxConnection*** with a single argument specifying the server name. For example, the following code retrieves an FTP connection:

```
CInternetSession session;
CFtpConnection* m_pFtp= session.GetFtpConnection("ftp.microsoft.com");
```

If the client computer is connected to the Internet through a proxy server, you need to specify the proxy server name, your log-on name, and your log-on password when making a connection.

The following example makes a request as "anonymous" through a proxy server called "ftp-gw." The user's e-mail name is specified as password.

```
CInternetSession session;
CFtpConnection* m_pFtp= session.GetFtpConnection("ftp-gw",
"anonymous@servername, myname@mycompany.com");
```

You must get the details about the proxy name from your network administrator.

CInternetFile

CInternetFile is the base class for the **CHttpFile** and **CGopherFile** file classes. It enables remote access to servers that use the HTTP and Gopher protocols. **CInternetFile** also is used when opening an FTP file.

CInternetFile objects are never created directly, but instead are created indirectly using **CGopherConnection::OpenFile**, **CHttpConnection::OpenRequest**, or **CFtpConnection::OpenFile**. For example:

```
// m_pFtp is a pointer to a CFtpConnection object.
CInternetFile* pFile = m_pFtp->OpenFile("readme.txt");
```

Getting Status Information about a Session

You can override the member function **CInternetSession::OnStatusCallback** to get status of your Internet session. Getting the status of the session is useful for lengthy operations, such as downloading a large file or searching the Web.

Notes

To get status of your session, you complete the following steps:

1. Derive a new class from **CInternetSession**.
2. Override the **CInternetSession::OnStatusCallback** member function.
3. Call **CInternetSession::EnableStatusCallback(TRUE)** to register the callback function.

To see sample code that shows how to implement **CInternetSession::OnStatusCallback**, click this icon.  (CD-ROM displays the following sample code.)

```
// Sample OnStatusCallback code for debugging.
void CInetSession::OnStatusCallback(
    DWORD dwContext, DWORD wInternetStatus,
    LPVOID lpvStatusInformation, DWORD dwStatusInformationLength)
{
    switch (wInternetStatus)
    {
    case INTERNET_STATUS_RESOLVING_NAME:
        TRACE("Looking up the IP address.\n");
        break;
    case INTERNET_STATUS_REQUEST_COMPLETE:
        TRACE("Asynchronous operation completed\n");
        break;
    case INTERNET_STATUS_HANDLE_CLOSING:
        TRACE("Handle closed.\n");
        break;
    case INTERNET_STATUS_HANDLE_CREATED:
        TRACE("New handle created.\n");
        break;
    case INTERNET_STATUS_NAME_RESOLVED:
        TRACE("IP address found.\n");
        break;
    case INTERNET_STATUS_CONNECTING_TO_SERVER:
        TRACE("Connecting to the socket address.\n");
        break;
    case INTERNET_STATUS_CONNECTED_TO_SERVER:
        TRACE("Successfully connected to the socket address.\n");
        break;
    case INTERNET_STATUS_SENDING_REQUEST:
        TRACE("Sending request to the server.\n");
        break;
    case INTERNET_STATUS_REQUEST_SENT:
        TRACE("Request sent to server.\n");
        break;
    case INTERNET_STATUS_RECEIVING_RESPONSE:
        TRACE("Waiting for server to respond.\n");
        break;
```

(Code continues on following page.)

Notes

```
    case INTERNET_STATUS_RESPONSE_RECEIVED:
        TRACE("Response received from server\n");
        break;
    case INTERNET_STATUS_CLOSING_CONNECTION:
        TRACE("Closing connection to server.\n");
        break;
    case INTERNET_STATUS_CONNECTION_CLOSED:
        TRACE("Connection to server closed.\n");
        break;
    default :
        TRACE("OnStatusCallback: Unknown status message\n");
        break;
    }
}
```

Notes

290 Writing HTTP Applications

Hypertext Transfer Protocol (HTTP) is an Internet protocol that has the following characteristics:

- Distributive

  One hypertext document may have links to many servers.
- Collaborative

  The client and server work together through the protocol to present the best possible presentation of the data to the user.
- Hypermedia

  The protocol is independent of the content being transferred.

HTTP sits on top of the TCP/IP layer, or another reliable transport layer, and is based on the request/response paradigm.

The Request/Respond Paradigm

HTTP clients and servers interact through messages. HTTP messages consist of client-to-server requests and server-to-client responses.

A request message consists of a request line, zero or more headers, and an optional entity-body. The request line has three parts: the request verb, the target URL, and the client's HTTP version number.

The following is a sample request message. It uses the GET verb to retrieve the desired target URL from the server.

```
GET http://www.microsoft.com/intdev/sdk/docs/wininet/ HTTP/1.0
Accept: text/*
User-Agent: MyClientProgram.exe
If-Modified-Since: Thu Oct24 10:07:04 1996
```

A response message consists of a status line, zero or more headers, and an optional entity-body. A status line consists of the server's HTTP version, a response code, and a short textual description of the code. The following is a sample response message:

```
HTTP/1.0 200 OK
Content-Type: text/html
Content-Length: 1277
Last-Modified: Tue, 15 Oct 1996 23:52:00 GMT
```

Processing a Hyperlink

When a user clicks a hyperlink within a hypertext document, the following sequence of events occurs:

1. The embedded hyperlink string is parsed for the server name, the path on the server, and the file name.
2. The client establishes a connection to a server.
3. The client sends a request, usually asking for a file.
4. The server responds with the data of that file.
5. The server closes the connection.

Notes

Establishing the Connection

To create an HTTP application, you must first create an Internet session by using the **CInternetSession** class. Once you create the session, you can use **CInternetSession::GetHttpConnection** to establish a connection.

The following example creates a session and establishes an HTTP connection to the microsoft.com Web site:

```
CInternetSession session;
CHttpConnection* pCon= session.GetHttpConnection("microsoft.com");
```

The **CHttpConnection** object manages an HTTP connection to a server.

Formulating a Request

Once you have established a connection, you can use **CHttpConnection::OpenRequest** to formulate a request to the Web server.

The **CHttpConnection** class has a single member function, **CHttpConnection::OpenRequest**, which retrieves a pointer to a **CHttpFile** object. You pass a verb and a resource to the **OpenRequest** function as shown in the following example:

```
CHttpFile* pFile=pCon->OpenRequest("GET","/intdev/sdk/docs/wininet/");
```

You can add headers to the message by using **CHttpFile::AddRequestHeaders**. You can add multiple headers by separating the headers using a carriage return/line feed (CR/LF) in the string parameter. For example, the following code adds two headers to the message:

```
CString strHeaders("Accept: text/*\r\nUser-Agent: MyClientProgram.exe\r\n");
pFile->AddRequestHeaders(strHeaders);
```

Sending a Request

Once you establish a connection and formulate a request, you can call **SendRequest** to send the request to the server as follows:

```
pFile->SendRequest();
```

Notes

Receiving a Response

You can use **CHttpFile::ReadString** to read the resultant data from the server, one line at a time, as shown in this example:

```
CString line;
while (pFile->ReadString(line)!=FALSE)
{
    //process a line of data from server.
}
```

To examine the response headers from the server, call **CHttpFile::QueryInfo**. You can extract individual header values, or you can extract all of the headers, as shown in this example:

```
CString strHeader;
if (pFile->QueryInfo(HTTP_QUERY_RAW_HEADERS_CRLF, strHeader))
{
    TRACE("%s\n", strHeader);
}
```

Notes

291 Writing FTP Applications

The File Transfer Protocol (FTP) enables transfer of files between incompatible computers, operating systems, and file systems. FTP also provides clients with general file management capabilities, such as retrieving directory listings, file attribute examination, directory navigation, and creation and deletion of directories on FTP servers.

MFC supports FTP mainly through the following classes:

- **CInternetSession**

  Manages your Internet session.
- **CFtpConnection**

  Manages your FTP connection and supports the directory-level FTP services.
- **CFtpFileFind**

  Enables you to work with file listings by examining file attributes.

The CFtpConnection Class

CFtpConnection manages your connection to the FTP server. You need to obtain a pointer to an instance of this class by using the function **CInternetSession::GetFtpConnection**.

Once you obtain this pointer, you can read and write files, move through the directory structure, and create and delete directories.

For a complete listing of **CFtpConnection** class members, search for "CFtpConnection" in MSDN Library Visual Studio 6.0.

The CFtpFileFind Class

CFtpFileFind is an MFC class derived from the **CFileFind** class. The **CFileFind** class supports wildcard characters and provides functions to retrieve file attributes. **CFtpFileFind** extends this base class by enabling searches and queries on files located on FTP servers.

CFtpFileFind supports UNIX-style searches using the wildcard characters asterisk (*****) and question mark (**?**). The asterisk wildcard specifies zero or more of any character, while the question mark specifies a single character.

For example, searching for files with the string **?rain*.?x?** will return matches on files such as "Drain.exe," "Train.txt," and "BrainMatter.wxy," but will not return a match on files such as "Rain.exe" and "Strain.exe."

CFtpFileFind also provides a rich selection of attribute functions, such as **GetCreationTime**, **IsDirectory**, and **IsHidden**.

To use the **CFtpFileFind** class, construct an instance of it. Then use **CFtpFileFind::FindFile** to initialize the search specifying a search criteria for the string parameter, with or without wildcard characters. You must call **CFtpFileFind::FindNextFile** before querying the attributes of the file found.

The search is performed in the FTP file server's current working directory. You can perform additional searches on other directories by using the same instance of **CFtpFileFind**, but you must reset the object by calling the **CFtpFileFind::Close** member function, changing the current remote directory, and then calling **CFtpFileFind::FindFile** again.

Notes

To view sample code that shows how to specify the search criteria and then iterates through each found file in one directory on the FTP server, click this icon.

 (CD-ROM displays the following sample code.)

```
//Sample code showing how to specify a search condition and process the
resultant data
CFtpFileFind ff(m_pFtp); // m_pFtp is a CFtpConnection* object.
BOOL success;
success= ff.FindFile("*");  // searching for all files, including directories,
in the current directory
while (TRUE ==success) {
// Need to call FindNextFile before doing any attribute queries.
    success =ff.FindNextFile();  // return value is for next file
//  ... perform any required file enquiry AFTER FindNextFile
    if (ff.IsDirectory())
//      ... do something with directories
    else
//      ... do something with files
}
ff.Close(); // close this search
```

Public FTP Servers

Generally, a client FTP application provides a user name and password to the FTP server. Often, though, public FTP servers are established to act as file servers to anyone connecting to them. To accomplish this, a convention has been established to use the string "anonymous" as the user name and the user's e-mail name as the password. Typically, the FTP server discards the password value.

To see the specifications of FTP, go to the FTP Specification page on the Network Working Group's Web site by clicking on this icon.

 (CD-ROM links to the Web site: ftp://nic.merit.edu/documents/rfc/rfc0959.txt.)

Notes

292 Writing Gopher Applications

The Gopher protocol supports distributed document search and retrieval. It is based on a menu-driven paradigm where the client is provided a list of menu items. Menu items can be a variety of resources, such as documents, directories, search engines, FTP sites, and sound files. When a user selects one of the menu items, the Gopher client retrieves the requested resource as a document, or accesses another Gopher server that then presents another list of menu items.

Using the Gopher protocol is like using HTTP. The client makes a connection with a Gopher server and sends a request. The server sends the information and then closes the connection.

For more information about the Gopher protocol, go to the Internet Gopher Protocol page on the following Network Working Group Web site by clicking on this icon.

 (CD-ROM links to the Web site: ftp://ds.internic.net/rfc/rfc1436.txt.)

MFC Support for Gopher

MFC provides general Internet support for the Gopher protocol through the **CInternetSession** class. In addition, the classes listed in the following table provide specific support for Gopher.

Class name	Description
CGopherConnection	This class manages the connection to the Gopher server. You use **CInternetSession::GetGopherConnection** to retrieve a pointer to an object.
CGopherLocatorType	This object type represents a token to a Gopher item. It is used by an application to retrieve the item.
CGopherFileFind	Similar to **CFtpFileFind**, this class responds to application requests to find specific documents.
CGopherFile	This class finds and reads files on a Gopher server.

Notes

293

Using the WinSock Classes

This section covers MFC support for low-level network programming. The section begins with a discussion of sockets and the TCP/IP protocol suite. This discussion is followed by an overview of the Windows Socket (WinSock) API, which is the standard API for network programming on Windows-based computers. The last topic in the section shows you how to use the **CAsyncSocket** and **CSocket** classes to create powerful Internet applications.

This section includes the following topics:

- Introduction to Sockets
- Using TCP/IP for Socket Communication
- The WinSock API
- MFC Support for WinSock

Notes

294 Introduction to Sockets

The term socket is used to describe an endpoint of communication on a computer network. A socket is associated with a process on the computer. A process needs a socket at its end, as well as at the other end of the network, to perform any communication. Most network interaction involves two sockets for peer-to-peer or client/server communication. Other configurations involve multicast or broadcast communication, such as the ones found in multiuser conferencing, radio broadcasting connections, and network management tasks.

The Socket API provides an abstraction layer to developers, helping you complete tasks that otherwise would require knowledge and manipulation of network card details and TCP/IP protocol details.

To use sockets for network communication, complete the following steps:

1. Create a local socket.
2. Connect with a remote computer.
3. Transfer data back and forth.
4. Close the connection.

Although using sockets eliminates some of the device- and protocol-specific programming work, network applications using sockets still must understand, and in some cases define, the format used for transferring the raw data. Therefore, writing applications using the WinSock classes requires more work than writing applications using the WinInet classes.

Types of Sockets

There are three types of sockets:

- Connection-oriented sockets

  A high-level socket that acts like a phone connection. Once a connection is established, it remains open until one of the parties explicitly ends it.

- Connectionless sockets

  A high-level socket that acts like a mail courier. It does not maintain a connection to the destination.

- Raw sockets

  A low-level socket that lets an application manipulate the individual fields of the protocol headers. For example, you can use raw sockets to create a network application that determines which routers a datagram passes through.

To see an animation that provides an overview of sockets and the types of sockets, click this icon.
 (CD-ROM plays the animation, "Programming with Sockets.")

Notes

The Roles of Clients and Servers in WinSock Programming

Clients and servers can be connection-oriented or connectionless. Connection-oriented servers require connection-oriented clients. Likewise, connectionless servers require connectionless clients.

- Role of a client application

  Initializes its socket and then initiates communication with a server. It is an active participant. A client application typically does not specify a local port number, but lets WinSock assign one for it.

- Role of a server application

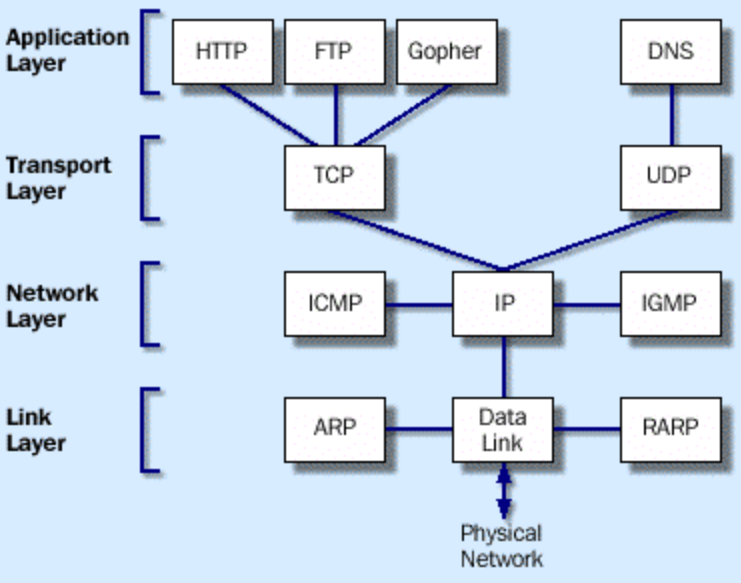
  Initializes its socket with a specific port number, and then waits passively for a client message. When a request comes in, the server application will respond by either creating a connection at a new port (if a streaming socket), or by accepting the datagram (if a datagram socket). A new port usually is created for connection-oriented sockets, so that the original port is kept open for other clients seeking connections.

Notes

295 Using TCP/IP for Socket Communication

The TCP/IP protocol suite is a set of protocols used to send data through the Internet. While the sockets paradigm is independent of the protocol suite used, TCP/IP is usually the underlying transport layer used for socket communication. Other protocols, such as FTP and HTTP, also use the TCP protocol as its underlying transport layer.

The TCP/IP suite is hierarchically structured. The following illustration shows the hierarchy of protocols in the suite. To see more information about a protocol, click its label in the illustration. (CD-ROM displays context-sensitive text.)



Protocols that provide sophisticated services depend on simpler protocols lower in the hierarchy. For instance, the Transmission Control Protocol (TCP) relies on a simpler network layer protocol, the Internet Protocol (IP) layer, which only attempts to deliver data to a destination computer. IP is an unreliable protocol. There is no guarantee provided at the IP layer.

The more sophisticated TCP layer provides a reliable communication connection between computers, guaranteeing that data will be successfully transferred. TCP guarantees the transfer of data by using techniques, such as handshaking, timeouts, retries, and checksum calculations.

Notes

296 The WinSock API

The Windows Socket API is a standard set of functions used on Windows-based computers to perform process-to-process communications across networks. It is based on the University of California, Berkeley, UNIX implementation BSD (Berkeley Software Distribution) version 4.3.

Windows Socket API Version 1.1

Windows Socket API version 1.1 consists of functions that can be divided into three functional groups: sockets, database, and Windows-specific.

Socket Functions

Some functions pertain to all socket types, while others pertain to only some types.

The following table lists some socket functions.

Function name	Description
accept	Acknowledges an incoming connection and associates it with an immediately created socket. The original socket returns to the listening state.
bind	Assigns a local name to an unnamed socket.
closesocket	Removes a socket from the per-process object reference table.
connect	Initiates a connection on the specified socket.

Database Functions

The following table lists some database functions.

Name	Description
gethostbyaddr	Retrieves the name(s) and address corresponding to a network address.
gethostbyname	Retrieves the name(s) and address corresponding to a host name.
gethostname	Retrieves the name of the local host.

Notes

Windows-Specific Functions

The following table lists some Windows-specific functions.

Name	Description
WSACancelBlockingCall	Cancels an outstanding blocking API call.
WSACleanup	Signs off from the underlying Windows Sockets DLL.
WSAGetLastError	Obtains details of the last Windows Sockets API error.
WSAIsBlocking	Determines whether the underlying Windows Sockets DLL is already blocking an existing call for this thread.

Windows Socket API Version 2

Windows Sockets version 2 provides many new features while maintaining backward compatibility with version 1.1. Architecturally, WinSock 2 is a layer separate from the transport layer (TCP, for example). In this respect, the WinSock 2 API has become more of an operating system component. Features of WinSock 2 include the following:

- Transport layer independence

  Windows Sockets 1.1 is TCP/IP-centric. WinSock 2.x uses the familiar sockets interface to provide access to protocols other than TCP/IP.
- Conventions for negotiating quality of service (QOS) levels for attributes like bandwidth and latency
- Protocol-independent multicast and broadcast

  Applications can discover what capabilities a transport layer provides.

To view the latest WinSocket 2.x specification, click this icon.

 (CD-ROM links to the Web site: http://www.stardust.com/wsresource/winsock2/readme.html.)

Notes

297 MFC Support for WinSock

MFC provides support for the Windows Socket API through two classes:

- **CSocket**

  This class encapsulates the API at a higher level and takes advantage of **CSocketFile** and **CArchive** to facilitate the sending and receiving of data.

- **CAsyncSocket**

  This class encapsulates the Windows Socket API at a lower level than **CSocket**. Use **CAsyncSocket** if you know network programming and want to program close to the API level.

For more information about these classes, search for "CSocket" and "ASyncSocket" in MSDN Library Visual Studio 6.0.

AfxSocketInit()

The Visual Studio Gallery provides a WinSock component that you can insert into your project so that your application can take advantage of the WinSock API. When you add the WinSock component to your project, a statement to call **AfxSocketInit** is placed in the **InitInstance** function of your application class.

By default, **AfxSocketInit** passes no parameter; however, you can pass a pointer to a WSADATA data structure to retrieve useful information about the Windows Socket API implementation on which your application is running. Information returned in this data structure includes the Windows Socket API version, the maximum number of sockets that it supports, and the maximum User Datagram Protocol (UDP) datagram size that your application can send.

Notes

299 Sample Applications

The following table provides a description of the sample application related to this chapter. The sample application is located in the folder \Samples\Ch09.

Sample application subfolder	Description of application
Browse	Implements an application with a **Web Browser** control.

For additional examples of Internet applications, refer to the MFC sample applications FTPTREE and TEAR, which are included in MSDN Library Visual Studio 6.0.

Notes

303

Self-Check Questions

1. The primary purpose of the CFtpFileFind class is to find which files?

- A. The first file that matches a search string on an FTP server
- B. All files in the current FTP directory that match the search string
- C. All files in the entire FTP server that match the search string
- D. All files on all active FTP servers that match the search string

2. Which classes are used by CSocket to simplify sending and receiving data using a socket?

- A. **CHttpConnection** and **CHtmlStream**
- B. **CAsyncSocket** and **COleStream**
- C. **CInternetFile** and **CInternetSession**
- D. **CSocketFile** and **CArchive**

3. Which type of socket should be established by the application to make communication between two computers reliable?

- A. Raw
- B. Connection-oriented (streaming)
- C. Datagram
- D. Connectionless

4. Which wrapper class does ClassWizard create for Internet Explorer object?

- A. **CWebBrowser2**
- B. **IWebBrowserApp**
- C. **CWinInet**
- D. **CWebBrowserApp**

Notes